

[Secure Exit](#)SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

CHITTIBOINA HARSHITHA
192473031RA

CO3-AT-Assignment

[View Only](#)[← Back](#)

QUESTIONS

Q1Banking Transaction System –
Exception Handling
10 marks**Q2**Hospital Patient Registration –
User-Defined Exception
10 marks**Q3**E-Commerce Order Processing
– Built-in Exceptions
10 marks**Q4**Online Food Delivery – Multiple
Threads and Execution Order
10 marks**Q5**Manufacturing Plant –
Producer-Consumer Problem
10 marks

5/5 answered

Q5

Manufacturing Plant – Producer-Consumer Problem

10
marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product.

Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:

- Make the producer wait when the buffer is full.
- Make the consumer wait when the buffer is empty.
- Prevent duplicate consumption.
- Ensure that products are consumed only after production.
- Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution

Consumer starts first Consumer waits until a product is

available Concurrency / design

Buffer capacity = 1; Products = P1, P2 Producer waits
while P1 remains unconsumed Exception / boundary

Your Code

```

1 import java.util.Scanner;
2 public class Main
3 {
4     static class Buffer
5     {
6         String product;
7         synchronized void produce(String p)
8             throws InterruptedException
9         {
10             while(product!= null)
11             {
12                 wait();
13             }
14             product = p;
15             System.out.println("produced:" +p)
16             notify();
17         }
18     }
19     synchronized void consume() throws Int
20     {
21         while(product == null)
22         {
23             wait();
24         }
25         System.out.println("consumed:" +pr
26         product = null;
27         notify();
28     }
29 }
30 public static void main(String[] args) th
31 {
32     Buffer b = new Buffer();
33     Thread producer = new Thread(() ->{
34         try
35         {
36             b.produce("p1");
37             b.produce("p2");
38             b.produce("p3");

```

```
39         }
40         catch (Exception e)
41         {
42             System.out.println(e);
43         }
44     });
45     Thread consumer = new Thread(() -> {
46         try
47         {
48             b.consume();
49             b.consume();
50             b.consume();
51         }
52         catch (Exception e)
53         {
54             System.out.println(e);
55         }
56     });
57     producer.start();
58     consumer.start();
59     producer.join();
60     consumer.join();
61 }
62 }
```

Input

stdin input

Output

produced:p1
consumed:p1
produced:p2
consumed:p2
produced:p3
consumed:p3

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.